

CentiSible — Documento Técnico

Relatório Técnico do Projeto de Prova de Aptidão Profissional (PAP)

Autor: António Santos
Curso: Programador de Informática (AP3-PROG18)
Ano Letivo: 2023–2026

Contexto: Relatório Final de Curso / Portefólio
Frontend: centisible.vercel.app
Backend API: centisible.onrender.com

Documento de referência do projeto. Descreve o que a aplicação faz, como está construída, que decisões foram tomadas e porquê, e que dificuldades surgiram durante o desenvolvimento. Complementa dois documentos já existentes: o `ARCHITECTURE.md` (desenho técnico detalhado, ERD, roadmap) e o `DEV_JOURNAL.md` (registo cronológico das decisões). Aqui a informação está consolidada e organizada por tema.

1. Identificação do Projeto

Campo	Valor
Nome	CentiSible (chamou-se FinTrack até 29 de agosto de 2026)
Tipo	Aplicação web de gestão de finanças pessoais e familiares, instalável como PWA
Contexto	Prova de Aptidão Profissional, 12.º ano. Serve também como peça de portefólio
Estado	Concluída e em produção, com contas de teste distribuídas a utilizadores reais
Endereços	Frontend em <code>centisible.vercel.app</code> , API em <code>centisible.onrender.com</code>
Repositório	<code>backend/</code> (FastAPI), <code>frontend/</code> (React), <code>docs/</code> (documentação)

2. Descrição Funcional

2.1 Problema que resolve

Muitas pessoas controlam as suas finanças numa folha de cálculo. Uma folha de cálculo regista os valores mas não faz nada com eles: não avisa quando um orçamento é ultrapassado, não projeta se um objetivo de poupança é atingível, não mostra a evolução dos últimos meses sem trabalho manual. O CentiSible substitui essa folha por uma aplicação que faz as contas e produz alertas de forma automática.

2.2 Utilizadores

A aplicação é multiutilizador. Cada pessoa regista-se, tem os seus próprios dados e nunca vê os dados de outra pessoa. Existe ainda o conceito de agregado familiar, que permite a duas pessoas juntarem as suas finanças numa vista partilhada, sempre mediante convite explícito.

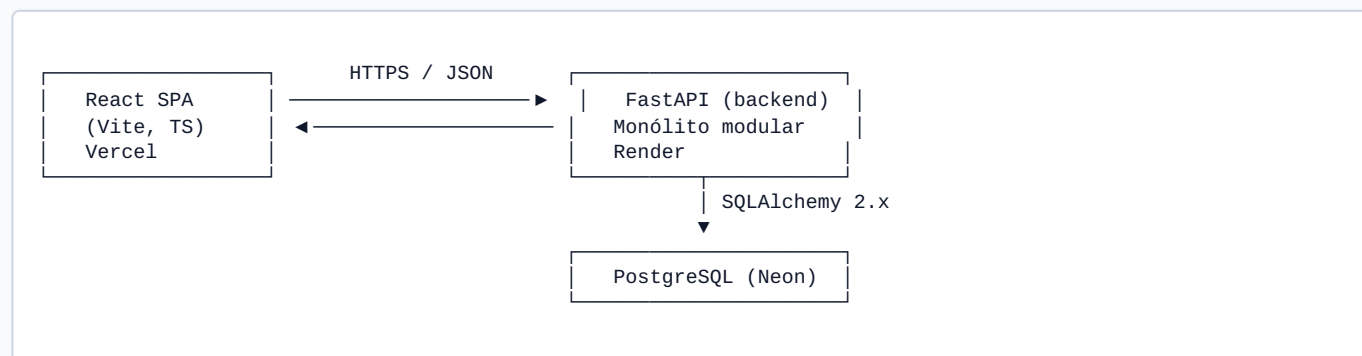
2.3 Funcionalidades

Área	Descrição
Autenticação	Registo, início de sessão, terminação de sessão, renovação automática da sessão e recuperação de palavra-passe por email.
Contas	Contas bancárias, carteiras, poupanças e cartões. Os cartões podem ter validade e plafond, com alerta quando a validade se aproxima ou o saldo desce abaixo do plafond.
Categorias	Categorias próprias de cada utilizador, com ícone e cor. Ao registar-se, o utilizador recebe um conjunto inicial de onze categorias para não começar com formulários vazios.
Transações	Receitas, despesas e transferências. Lista agrupada por dia, filtros por conta, categoria, tipo e intervalo de datas, recibo anexável em foto ou PDF, e exportação para CSV compatível com Excel português.
Orçamentos	Limite mensal por categoria, com cálculo automático de gasto, restante e percentagem, apresentado com barra de progresso.
Objetivos	Meta de poupança com prazo, contribuições e cálculo de quanto seria necessário poupar por mês para a cumprir.
Despesas recorrentes	Definição de despesas periódicas, mensais ou anuais, com um mecanismo que gera as transações em falta quando invocado.
Dashboard	Resumo do mês selecionado: saldo total, receitas, despesas, taxa de poupança, gráfico de despesas por categoria e próximos pagamentos.
Histórico e análise	Comparação com o mês anterior e evolução de seis ou doze meses.
Insights	Regras automáticas que produzem alertas sobre variações de despesa, orçamentos ultrapassados, projeções de objetivos e cartões.
Agregado familiar	Criação de um agregado, convite a outro utilizador por email, aceitação ou recusa, e alternância do dashboard entre vista individual e vista do agregado.
PWA	Aplicação instalável em telemóvel e computador, com manifesto, ícones e service worker.
Tema	Modo claro e escuro, com alternador manual e persistência da preferência.

3. Arquitetura

3.1 Visão geral

O CentiSible é composto por duas aplicações independentes que comunicam apenas através de uma API REST sobre HTTPS, trocando JSON.



O frontend nunca comunica diretamente com a base de dados. O backend é o único componente que o faz.

Optou-se por um monólito modular e não por microserviços. A complexidade do domínio, que envolve dinheiro, orçamentos e recorrências, é suficiente para demonstrar competência em arquitetura sem introduzir os custos de um sistema distribuído, que não se justificam numa aplicação usada por um utilizador de cada vez e sem picos de tráfego.

3.2 Camadas do backend

O backend está organizado em camadas com uma regra simples: cada camada só chama a camada imediatamente abaixo, nunca ao contrário e nunca saltando uma.

```
api/      routers FastAPI. Recebem o pedido, validam a forma, identificam
          o utilizador, chamam o serviço, confirmam a transação e devolvem JSON.
          Não decidem regras de negócio.

schemas/  modelos Pydantic de entrada e saída. Validação automática dos pedidos.

services/ toda a lógica de negócio. Exemplos: uma transferência não conta como
          despesa nos totais, repor a palavra-passe revoga todas as sessões,
          o saldo da conta é atualizado no mesmo momento que a transação.

repositories/ acesso a dados. Funções curtas com as queries SQLAlchemy.
              Filtram sempre por user_id, o que constitui a base do isolamento
              entre utilizadores.

models/    modelos ORM SQLAlchemy. As doze tabelas e as relações entre elas.

db/        engine, sessão por pedido, classe base declarativa.

core/      configuração, segurança, exceções, rate limiting, logging, envio
          de email, aritmética de datas.
```

Um router chama um serviço. Um serviço usa um ou mais repositórios e pode compor outros serviços. Um repositório apenas sabe executar queries, não toma decisões. A frase "uma transferência não conta como despesa" vive no serviço, não no repositório.

3.3 Tratamento de erros

Todas as exceções de negócio herdam de uma classe base `DomainError`, definida em `core/exceptions.py`, que transporta o código HTTP e a mensagem a devolver. Um único handler registado em `main.py` traduz qualquer uma delas numa resposta JSON. Os routers não capturam exceções, limitam-se a chamar o serviço e a confirmar a transação. A única exceção a esta regra é o endpoint de renovação de sessão, onde a resposta de erro também manipula o cookie, o que exige tratamento explícito.

3.4 Organização do frontend

O frontend está organizado por funcionalidade e não por tipo de ficheiro.

routes/	uma página por ficheiro. Páginas públicas (landing, login, registo, recuperação de palavra-passe) e páginas protegidas (dashboard, contas, categorias, transações, orçamentos, objetivos, recorrentes, histórico, agregado, definições).
features/	lógica por domínio. Cada área tem as chamadas à API, os tipos dos dados e os schemas de validação Zod. A área de autenticação contém também o contexto de sessão.
api/	client.ts é o único ponto que comunica com o backend. Anexa o token de acesso, renova a sessão quando recebe 401, tenta o pedido de novo e serializa as renovações entre separadores do browser.
components/	componentes reutilizáveis, incluindo os blocos base ao estilo shadcn/ui que estão copiados para dentro do repositório.
lib/	utilitários puros: formatação de valores, aritmética de meses, composição de classes CSS.

O estado do servidor é gerido pelo TanStack Query, que trata do cache, dos estados de carregamento e erro, e do refetch. O frontend não guarda dados de forma persistente, apenas um cache e a sessão.

4. Tecnologias

As tecnologias foram escolhidas por serem padrão da indústria, relevantes para um percurso profissional em desenvolvimento, e ao mesmo tempo explicáveis em detalhe numa defesa oral. Sempre que uma opção mais sofisticada não acrescentava valor de aprendizagem proporcional ao custo de a explicar, foi deixada de fora e documentada como extensão futura.

4.1 Backend

Tecnologia	Função	Justificação
Python 3.12	Linguagem	Requisito do projeto. Ecossistema maduro para APIs e para trabalho com dados.
FastAPI	Framework web	Validação automática dos pedidos via Pydantic, documentação OpenAPI gerada sozinha, suporte assíncrono. Django traria componentes não necessários, Flask exigiria montar manualmente o que o FastAPI já dá.
SQLAlchemy 2.x	ORM	Estilo moderno com anotações de tipo. Queries em Python, protegidas contra SQL injection por construção.
Alembic	Migrações	Alterações à estrutura da base de dados versionadas e aplicadas por ordem, também em produção.
Pydantic v2	Schemas	Validação e serialização de entrada e saída da API.
PostgreSQL	Base de dados	Transações ACID, que são indispensáveis quando se mexe em dinheiro, tipo <code>NUMERIC</code> para aritmética exata, chaves estrangeiras e enums nativos. É também a base mais pedida em vagas de backend júnior.
bcrypt	Hash de passwords	Usado diretamente. A biblioteca <code>passlib</code> , prevista inicialmente, não tem lançamentos desde 2020 e é incompatível com versões atuais do <code>bcrypt</code> .
PyJWT	Tokens de sessão	

Tecnologia	Função	Justificação
		Mais simples e mais ativamente mantida do que a alternativa <code>python-jose</code> .
slowapi	Rate limiting	Limita pedidos por IP nos endpoints de autenticação.
python-multipart	Upload de ficheiros	Necessária para receber os recibos anexados às transações.
uv	Gestor de pacotes	Rápido, com ficheiro de bloqueio de versões.

O envio de email para a recuperação de palavra-passe usa a API HTTP da Resend, invocada com o módulo `urllib` da biblioteca padrão, sem introduzir qualquer dependência nova.

4.2 Frontend

Tecnologia	Função	Justificação
React 19	Biblioteca de UI	Padrão da indústria para SPAs.
TypeScript	Linguagem	Apanha erros de tipo antes da execução.
Vite	Build e Dev Server	Arranque rápido, configuração simples, suporte nativo a PWA através de plugin.
React Router	Routing	Navegação do lado do cliente, com rota de layout para a barra lateral não voltar a montar a cada clique.
TanStack Query	Estado de servidor	Evita reescrever a lógica de cache, carregamento, erro e refetch em cada página.
React Hook Form + Zod	Formulários	Formulários performantes com validação por schema, partilhável entre cliente e definição de tipos.
Tailwind CSS	Estilos	Estilização utilitária, rápida e consistente.
shadcn/ui	Componentes base	Botões, cartões, modais e inputs baseados em Radix, copiados para dentro do repositório. Não é uma dependência de runtime, o que dá controlo total sobre cada componente.
Motion	Animações	Transições de página e micro-interações. Padrão de facto em React para este tipo de acabamento.
Recharts	Gráficos	Biblioteca de gráficos mais usada no ecossistema React, com animações incluídas.
vite-plugin-pwa	PWA	Gera o manifesto e o service worker. Os pedidos à API nunca passam por cache, os dados financeiros vêm sempre da rede.

4.3 Infraestrutura e qualidade

Tecnologia	Função
Docker Compose	Sobe base de dados, backend e frontend com um comando, tanto em desenvolvimento como numa referência local de produção.
GitHub Actions	Integração contínua. Seis jobs por push: lint do backend, lint do frontend, testes do backend, build do frontend, testes end-to-end e build das imagens.
pytest	Testes do backend, executados contra um PostgreSQL real.

Tecnologia	Função
Playwright	Testes end-to-end que usam a aplicação como um utilizador.
ruff	Lint e formatação do Python.
oxlint	Lint do TypeScript.
Vercel, Render, Neon, Resend	Alojamento do frontend, do backend, da base de dados e do envio de email. Todos com plano gratuito real, sem cartão de crédito.

5. Modelo de Dados

5.1 Convenções

Todas as tabelas têm `id` do tipo UUID como chave primária, e as colunas `created_at` e `updated_at` com fuso horário. Todas as tabelas de domínio têm uma coluna `user_id`, com chave estrangeira e índice, que é a base da autorização ao nível dos dados: qualquer query filtra por este campo.

A escolha de UUID em vez de inteiro sequencial dá identificadores não adivinháveis, geráveis no cliente sem ida ao servidor e fáceis de fundir entre ambientes. O custo em espaço de índice é irrelevante nesta escala.

5.2 Tabelas

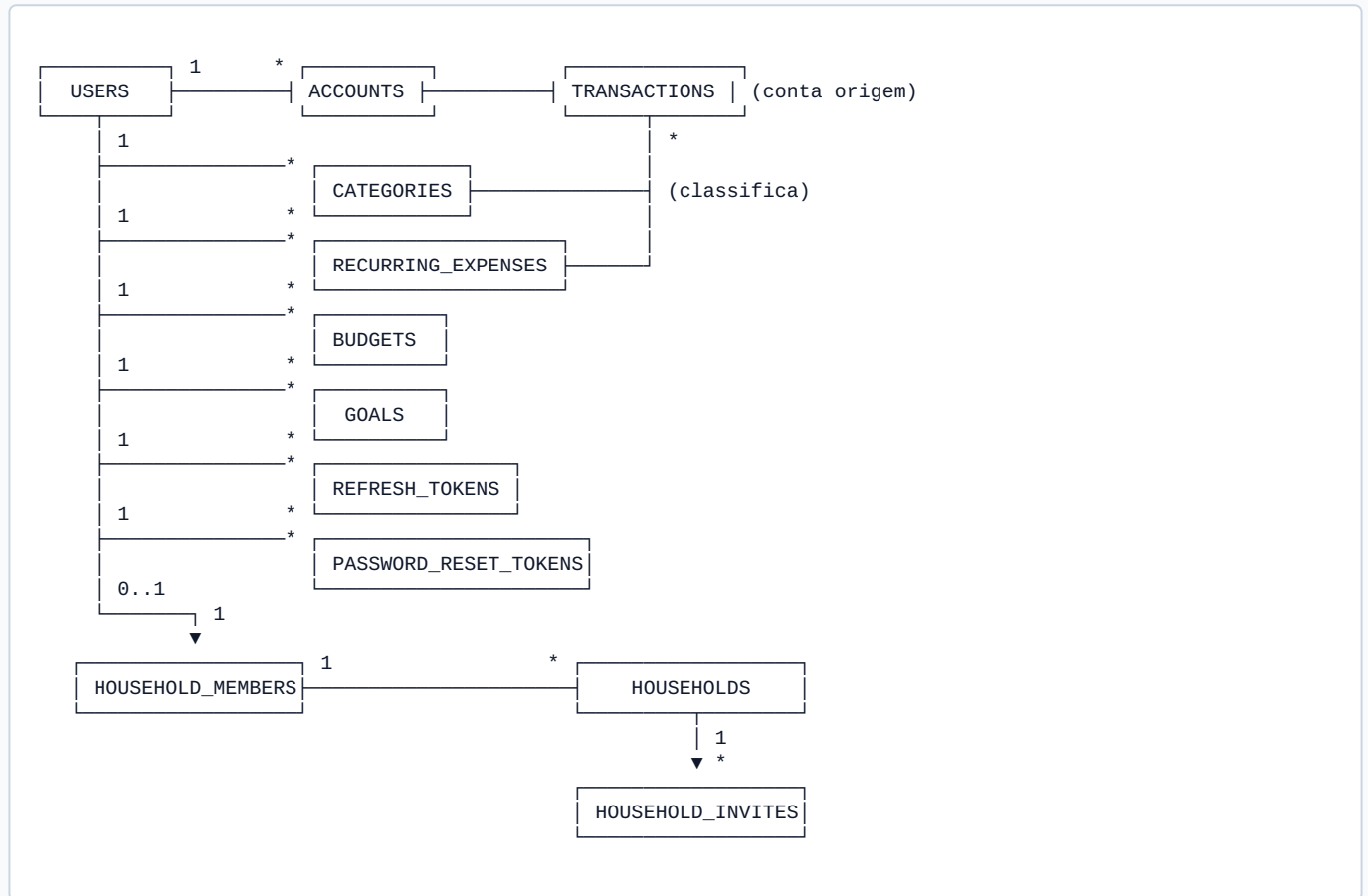
- **users:** email único, hash da palavra-passe, nome, moeda com valor por omissão `EUR` e rendimento mensal opcional.
- **accounts:** pertencem a um utilizador. Tipo entre banco, carteira, poupança, cartão de crédito e outro. Saldo inicial e saldo atual, este último mantido consistente pela camada de serviço. Validade e plafond opcionais, usados apenas nos cartões pré-pagos para gerar alertas.
- **categories:** pertencem a um utilizador. Têm um tipo, receita ou despesa. O enunciado descrevia categorias apenas para despesas, mas uma receita também beneficia de categoria, por exemplo "Salário" ou "Freelance". O tipo permite que os orçamentos só façam sentido em categorias de despesa e que a interface mostre as categorias certas em cada formulário. Existe uma restrição de unicidade sobre o par utilizador e nome.
- **transactions:** a tabela central. Pertence a um utilizador, tem uma conta de origem obrigatória, uma conta de destino usada apenas nas transferências, e uma categoria opcional, nula nas transferências. Tipo entre receita, despesa e transferência. O valor é sempre positivo, com uma restrição `CHECK`, e o sinal resulta do tipo, não do número. Tem uma data indexada, um campo booleano `is_shared` que marca despesas do agregado familiar, e uma referência ao recibo anexado, cujo ficheiro vive em disco e nunca é servido por URL pública. Uma restrição `CHECK` garante que uma transferência tem conta de destino e não tem categoria.
- **recurring_expenses:** modelo de uma despesa periódica, com categoria, conta, valor, frequência mensal ou anual, dia do mês, data da próxima ocorrência indexada e um campo de ativação.
- **budgets:** orçamento de uma categoria para um mês, identificado pelo primeiro dia desse mês, o que simplifica as queries por período. O gasto, o restante e a percentagem não são colunas, são calculados em tempo de execução a partir das transações do mês. Guardar valores derivados criaria uma segunda fonte de verdade que poderia ficar desatualizada. Existe uma restrição de unicidade sobre utilizador, categoria e mês.
- **goals:** objetivo de poupança com valor alvo, valor atual, prazo opcional e contribuições.
- **refresh_tokens:** suporta a terminação de sessão real e a rotação e revogação de tokens de renovação. Guarda o hash do token, nunca o token em claro, a data de expiração, um campo de revogação e a data em que foi revogado.
- **households, household_members, household_invites:** modelam o agregado familiar. Um agregado tem um nome e um criador. A tabela de membros tem uma restrição de unicidade sobre `user_id`, e não sobre o par agregado e utilizador, o que garante que uma pessoa só pertence a um agregado de cada vez e elimina qualquer ambiguidade no alternador da interface. A tabela de convites tem um estado entre pendente, aceite, recusado e cancelado, e um

índice único parcial que impede convites pendentes duplicados para a mesma pessoa. Ao aceitar um convite, o serviço cria a linha de membro dentro da mesma transação.

- **password_reset_tokens:** guarda o hash SHA-256 do token de reposição, com prazo de uma hora, uso único registado num campo `used_at`, e a regra de que um novo pedido invalida os anteriores.

No total são doze tabelas, com migrações Alembic aplicadas por ordem.

5.3 Diagrama de Entidades (ERD)



Nota: A conta de destino das transferências é também uma chave estrangeira para a tabela de contas.

5.4 Regras de integridade referencial

As relações a partir da tabela de utilizadores usam `ON DELETE CASCADE`. Se um utilizador for apagado, os seus dados vão com ele, o que é razoável para dados pessoais.

As relações de categorias e contas para transações usam `ON DELETE RESTRICT`. Apagar uma categoria não deve apagar transações em silêncio. Se existirem transações associadas, a API responde com `409 Conflict` e uma mensagem clara, e o frontend oferece reatribuir as transações a outra categoria antes de eliminar.

6. API

A API está publicada sob o prefixo `/api/v1`. A documentação interativa OpenAPI está disponível em `/docs`. Todos os endpoints exigem autenticação, exceto os de registo, início de sessão, renovação e recuperação de palavra-passe.

6.1 Autenticação (`/auth`)

Método e caminho	Descrição
POST <code>/auth/register</code>	

Método e caminho	Descrição
	Cria um utilizador, cria o conjunto inicial de categorias e devolve o par de tokens. Limitado a 10 pedidos por minuto.
POST /auth/login	Valida credenciais e devolve o par de tokens. Limitado a 10 pedidos por minuto.
POST /auth/refresh	Recebe o refresh token pelo cookie, rota-o e devolve um novo access token.
POST /auth/logout	Revoga o refresh token e limpa o cookie.
POST /auth/password-reset/request	Pede a reposição da palavra-passe. Responde sempre com 202 e a mesma mensagem, não revela se o email existe. Limitado a 5 pedidos por hora.
POST /auth/password-reset/confirm	Confirma a reposição com o token recebido por email. Limitado a 10 pedidos por hora.

6.2 Utilizador (/users)

Método e caminho	Descrição
GET /users/me	Devolve o perfil do utilizador autenticado.
PATCH /users/me	Atualiza nome, moeda e rendimento mensal.

6.3 Contas (/accounts)

Método e caminho	Descrição
GET /accounts	Lista as contas do utilizador com o saldo atual.
POST /accounts	Cria uma conta.
PATCH /accounts/{id}	Atualiza uma conta.
DELETE /accounts/{id}	Elimina uma conta.

6.4 Categorias (/categories)

Método e caminho	Descrição
GET /categories	Lista as categorias do utilizador.
POST /categories	Cria uma categoria.
PATCH /categories/{id}	Atualiza nome, tipo, ícone ou cor.
DELETE /categories/{id}	Elimina uma categoria. Aceita <code>reassign_to_category_id</code> para reatribuir as transações antes de eliminar. Responde 409 se houver transações e nenhuma reatribuição.

6.5 Transações (/transactions)

Método e caminho	Descrição
GET /transactions	Lista com filtros por conta, categoria, tipo e intervalo de datas.
GET /transactions/export	Exporta as transações filtradas para CSV com separador ponto e vírgula, vírgula decimal e BOM, compatível com Excel português.

Método e caminho	Descrição
POST /transactions	Cria uma transação e atualiza o saldo da conta no mesmo commit. Aceita <code>allow_duplicate</code> para confirmar uma despesa partilhada sinalizada como possível duplicado.
PATCH /transactions/{id}	Atualiza uma transação e reconcilia o saldo. Aceita <code>allow_duplicate</code> .
DELETE /transactions/{id}	Elimina uma transação e reverte o efeito no saldo.
POST /transactions/{id}/receipt	Anexa um recibo em foto ou PDF.
GET /transactions/{id}/receipt	Descarrega o recibo, servido apenas ao dono.
DELETE /transactions/{id}/receipt	Remove o recibo.

6.6 Orçamentos (/budgets)

Método e caminho	Descrição
GET /budgets	Lista os orçamentos do mês indicado, com gasto, restante e percentagem calculados.
POST /budgets	Cria um orçamento para uma categoria e um mês.
PATCH /budgets/{id}	Atualiza o valor.
DELETE /budgets/{id}	Elimina o orçamento.

6.7 Despesas recorrentes (/recurring-expenses)

Método e caminho	Descrição
GET /recurring-expenses	Lista as despesas recorrentes.
POST /recurring-expenses	Cria uma despesa recorrente.
POST /recurring-expenses/generate	Percorre as recorrências ativas com ocorrência em atraso, cria as transações em falta e avança a data da próxima ocorrência.
PATCH /recurring-expenses/{id}	Atualiza uma recorrência.
DELETE /recurring-expenses/{id}	Elimina uma recorrência.

6.8 Objetivos (/goals)

Método e caminho	Descrição
GET /goals	Lista os objetivos com a projeção de conclusão.
POST /goals	Cria um objetivo.
PATCH /goals/{id}	Atualiza um objetivo.
POST /goals/{id}/contributions	Regista uma contribuição para o objetivo.
DELETE /goals/{id}	Elimina um objetivo.

6.9 Dashboard, análise e insights

Método e caminho	Descrição
GET /dashboard	Resumo do mês. Aceita <code>month</code> e <code>scope</code> (<code>individual</code> ou <code>household</code>).
GET /analytics/monthly-comparison	Compara o mês indicado com o mês anterior.
GET /analytics/monthly-trend	Evolução de 2 a 24 meses, por omissão 6.
GET /insights	Lista de alertas automáticos do mês.

6.10 Agregado familiar (/households)

Método e caminho	Descrição
POST /households	Cria um agregado.
GET /households/me	Devolve o agregado do utilizador e os seus membros.
POST /households/me/leave	Sai do agregado.
POST /households/me/invites	Convida outro utilizador por email.
GET /households/me/invites	Lista os convites enviados.
DELETE /households/me/invites/{id}	Cancela um convite enviado.
GET /households/invites	Lista os convites recebidos.
POST /households/invites/{id}/accept	Aceita um convite e passa a membro.
POST /households/invites/{id}/decline	Recusa um convite.

7. Autenticação e Segurança

7.1 Modelo de sessão

A sessão assenta em dois tokens.

O **access token** é um JWT de curta duração, quinze minutos, devolvido no corpo da resposta e guardado apenas em memória no frontend, num contexto React. Nunca é escrito em `localStorage`, o que reduz o impacto de um ataque de XSS.

O **refresh token** tem duração longa, trinta dias. É um token opaco de alta entropia, gerado com `secrets.token_ur1safe`, não um JWT. É entregue num cookie `httpOnly` e `Secure` fora de desenvolvimento, com o caminho limitado aos endpoints de autenticação, pelo que só é enviado nos pedidos de sessão e nunca nos restantes pedidos à API. Na base de dados é guardado como hash SHA-256. Usa-se SHA-256 e não bcrypt porque se trata de um segredo aleatório de 512 bits e não de uma palavra-passe humana. Não há força bruta a mitigar, apenas o risco de uma fuga da tabela.

Quando o access token expira, o frontend deteta o `401`, chama o endpoint de renovação com o cookie e repete o pedido original. O utilizador não se apercebe.

7.2 Rotação e deteção de reutilização

A cada renovação o refresh token antigo é revogado e é emitido um novo. Reutilizar um token já rodado indica normalmente roubo, e nesse caso toda a família de tokens do utilizador é revogada.

Existe uma exceção: Se o token foi rodado há menos de dez segundos, o pedido é tratado como uma corrida benigna, por exemplo o mesmo cookie enviado quase em simultâneo pela aplicação instalada e por um separador do browser, e o backend responde `409` sem revogar nada. Esta abordagem segue a RFC 9700, a boa prática de segurança para OAuth 2.0. O cliente reforça ainda a proteção serializando as renovações entre separadores e garantindo um único pedido de renovação em curso de cada vez.

7.3 Isolamento entre utilizadores

Todas as queries filtram por `user_id`. Se um utilizador pedir um recurso que existe mas pertence a outra pessoa, a resposta é `404` e não `403`, de forma deliberada, para não confirmar sequer que o identificador existe. Existe uma pasta de testes dedicada a verificar que nenhum endpoint expõe dados de outro utilizador.

7.4 Outras medidas

- Rate limiting por IP nos endpoints de registo, início de sessão e recuperação de palavra-passe.
- Cabeçalhos de segurança HTTP em todas as respostas: `X-Content-Type-Options`, `X-Frame-Options: DENY`, `Referrer-Policy`, `Cross-Origin-Opener-Policy`, `Permissions-Policy` e uma Content Security Policy mínima. O `Strict-Transport-Security` só é enviado em produção.
- No frontend, uma Content Security Policy restritiva definida numa fonte única, lida tanto pela configuração de produção como pelo servidor de desenvolvimento, para que as violações apareçam cedo.
- A aplicação recusa-se a arrancar em produção se a chave secreta não estiver definida.
- Logging estruturado em JSON dos erros do backend.
- Limpeza periódica da tabela de refresh tokens expirados.

8. Decisões Técnicas Relevantes

Dinheiro em `NUMERIC` e `Decimal`: Nenhum cálculo monetário usa vírgula flutuante. O PostgreSQL guarda `NUMERIC(12,2)` e o Python trabalha com `Decimal`. A alternativa de guardar tudo em cêntimos como inteiro é válida e usada por alguns sistemas de pagamento, mas o par `NUMERIC` e `Decimal` evita a conversão mental para cêntimos em cada camada e é o padrão na maioria das aplicações financeiras de porte médio. No frontend os valores chegam como texto seguro e são apenas formatados para apresentação, nunca somados em JavaScript quando o resultado tem de estar correto. Esses totais vêm sempre calculados no backend.

Saldo guardado, não recalculado a cada pedido: O saldo atual de uma conta é uma coluna, atualizada sempre que uma transação é criada, editada ou eliminada, dentro do mesmo commit. É mais rápido de ler e todos os caminhos que mexem em transações passam pelo mesmo código, pelo que o saldo não dessincroniza.

Valores derivados dos orçamentos calculados em tempo de execução: Ao contrário do saldo, o gasto e o restante de um orçamento não são colunas. São baratos de calcular a partir das transações do mês e guardá-los criaria uma fonte de verdade duplicada sem ganho real.

Agregado familiar como camada de leitura: As contas e as transações pertencem sempre ao utilizador individual. Não existe conta conjunta na base de dados. Quando o dashboard é pedido em modo agregado, o serviço identifica todos os membros e soma os dados de todos. A vantagem é que o modelo de contas e transações não muda, e cada pessoa mantém o histórico e a privacidade dos seus registos. Juntar-se a um agregado exige aceitar um convite explícito, porque implica expor dados financeiros a outra pessoa.

Despesas partilhadas no modelo "lançada uma vez": Uma despesa marcada como partilhada é um custo da casa, por exemplo a renda, lançado uma única vez por quem pagou. Na vista do agregado conta uma vez. Para evitar o lançamento em dobro, quando outro membro já tem uma despesa partilhada com a mesma categoria e o mesmo valor no mesmo mês, a criação é recusada com `409` e uma mensagem que nomeia a pessoa. O utilizador pode confirmar e reenviar com `allow_duplicate`. Valores diferentes na mesma categoria, por exemplo um casal que divide a renda em 500 e 300, somam-se normalmente. Não há dedução automática nem acerto de contas ao estilo Splitwise.

Categoria com tipo: Adicionar o tipo receita ou despesa à categoria resolve uma ambiguidade real do enunciado e permite filtrar as categorias certas em cada formulário e restringir os orçamentos a categorias de despesa.

Repository pattern leve: Os repositórios são funções que encapsulam queries por entidade, não uma abstração genérica. O objetivo é separar como obter os dados de o que fazer com eles, e manter os serviços testáveis.

Testes de integração sem Testcontainers: Os testes correm contra um PostgreSQL real disponibilizado pelo Docker Compose em local e por um serviço nativo no GitHub Actions. Continua a ser Postgres real e não SQLite, o que evita bugs que o SQLite mascara, mas dispensa mais uma biblioteca a explicar.

End-to-end com âmbito reduzido: O Playwright cobre um conjunto de fluxos críticos, como registo, criação de transação, criação de orçamento e carregamento do dashboard, em vez de uma suite exaustiva. É suficiente para demonstrar competência sem consumir uma fatia desproporcional do tempo.

Alojamento gratuito: Frontend na Vercel, backend na Render a partir de um Dockerfile de produção, e base de dados na Neon. Os três têm plano gratuito real. A Fly.io foi a primeira escolha mas o seu plano gratuito deixou de existir em 2024.

9. Dificuldades Encontradas

Esta secção documenta problemas concretos surgidos durante o desenvolvimento e a forma como foram resolvidos. São exemplos úteis para explicar o processo, não apenas o resultado.

A sessão morria logo após o início de sessão em produção: Depois do primeiro deploy real, o utilizador era autenticado e perdia a sessão no pedido seguinte. A causa era o atributo `SameSite` do cookie do refresh token, que estava em `Lax`. Um cookie `Lax` nunca é enviado pelo browser em pedidos entre sites diferentes. Em desenvolvimento local o frontend e o backend só diferem na porta, por isso funcionava. Em produção vivem em domínios completamente diferentes, a Vercel e a Render, e o cookie deixava de ser enviado. A correção foi usar `SameSite=None` em produção, o que obriga a `Secure`, condição que já estava garantida apenas em produção.

"Não autenticado" de forma intermitente ao criar contas ou cartões: A deteção de reutilização de refresh tokens era demasiado agressiva. Dois pedidos de renovação quase simultâneos, situação normal com a aplicação instalada aberta ao mesmo tempo que um separador do browser, ou um F5 durante um pedido lento, eram interpretados como roubo e revogavam toda a família de tokens. A solução foi introduzir uma janela de tolerância de dez segundos, durante a qual a reutilização é tratada como corrida benigna e devolve `409` sem revogar nada, e serializar as renovações no cliente com `navigator.locks`.

A biblioteca de hash de palavras-passe não funcionava: A escolha inicial era `passlib` com backend `bcrypt`. Na prática, o `passlib`, que não tem lançamentos desde 2020, é incompatível com as versões atuais do `bcrypt` e falhava ao arrancar. A solução foi usar o `bcrypt` diretamente, através de `hashpw` e `checkpw`, que cobre as duas funções necessárias sem a camada intermédia.

Os emails de recuperação de palavra-passe não chegavam: A API da Resend está atrás da Cloudflare, que bloqueava o User-Agent por omissão do cliente HTTP do Python. A correção foi definir um User-Agent explícito no pedido.

O formulário de recarga de plafond fechava-se sozinho: Ao abrir a página de transações já com o formulário preenchido, vindo de um botão nos cartões pré-pagos, o formulário abria e fechava um instante depois. A causa era uma chamada de navegação dentro de um `useEffect` a interagir mal com a animação de saída da rota protegida. Foi resolvido movendo a navegação para fora do efeito.

Uma classe de largura máxima nunca tinha efeito: No frontend, aplicar `mx-auto` sem `w-full` fazia com que a largura máxima definida num contentor nunca fosse respeitada. Um bug de compreensão de flexbox, corrigido acrescentando a largura.

Retângulo preto ao tocar num gráfico em telemóvel: O Recharts não desliga por omissão o realce de toque do browser, o que produzia um retângulo preto sobre o gráfico. Resolvido com a propriedade CSS adequada.

Configuração da Render antes do primeiro deploy limpo: Foram necessários dois ajustes de configuração na Render antes de o backend arrancar corretamente em produção, ambos documentados no diário de desenvolvimento.

Fusão incorreta de despesas por categoria no agregado: A lista de despesas por categoria na vista do agregado fundia linhas com o mesmo nome de forma cega, misturando despesas pessoais homônimas de pessoas diferentes. Foi corrigido para só fundir despesas efetivamente marcadas como partilhadas, mantendo as despesas pessoais separadas por dono.

10. Qualidade, Testes e Integração Contínua

O backend tem cerca de 250 testes, organizados em testes unitários para peças isoladas como datas e regras de recorrência, testes de API que exercitam cada endpoint de ponta a ponta contra um PostgreSQL real, e testes de segurança dedicados a autenticação, autorização e injeção de SQL. Cada teste corre dentro de uma transação que é sempre revertida no fim, pelo que os testes nunca deixam dados na base.

O frontend tem uma dúzia de testes end-to-end em Playwright que percorrem os fluxos críticos usando a aplicação como um utilizador. Estes testes apanharam pelo menos um bug de concorrência que os testes unitários não revelavam.

A integração contínua no GitHub Actions executa seis jobs a cada push: lint do backend, lint do frontend, testes do backend, build do frontend, testes end-to-end e build das imagens Docker. Todos verdes.

11. Implementação e Alojamento

Em desenvolvimento, `docker compose up -d` sobe três contentores: PostgreSQL, backend com recarga automática e frontend com o servidor Vite. A aplicação fica em `localhost:5173` e a API em `localhost:8000`, com a documentação em `/docs`.

Em produção existem três serviços separados. O frontend é um build estático na Vercel. O backend corre na Render a partir de um Dockerfile de produção, sem recarga automática e sem o gestor de pacotes na imagem final. A base de dados é um PostgreSQL gerido na Neon. As migrações Alembic são aplicadas no deploy.

O trade-off do plano gratuito da Render é que o backend adormece ao fim de quinze minutos sem pedidos e demora cerca de trinta segundos a acordar no primeiro pedido seguinte.

12. Limitações Conhecidas e Trabalho Futuro

- **As despesas recorrentes não se geram sozinhas:** O endpoint de geração só corre quando o utilizador carrega no botão correspondente. Não há tarefa agendada a chamá-lo. É uma decisão em aberto: aceitar como passo manual esperado ou ligar a geração a um agendador.
- **Os recibos não persistem em produção:** O plano gratuito da Render não oferece disco persistente, pelo que um recibo anexado não sobrevive a um reinício do contentor. Em desenvolvimento, com o volume do Docker Compose, o problema não existe. A resolução prevista é mover o armazenamento para uma coluna binária na base de dados, com um limite de tamanho mais baixo.
- **A limpeza de refresh tokens é menos fiável em produção:** Corre uma vez a cada 24 horas dentro do próprio processo. Na Render gratuita, que adormece, um ciclo contínuo de 24 horas raramente se completa. O impacto é baixo na escala atual.
- **Falta a confirmação de email no registo:** Agora que o envio de email existe, é um acréscimo pequeno.
- **Outras extensões possíveis:** scan de dependências no CI, autenticação em dois fatores, monitorização de erros em produção, exportação do resumo mensal em PDF, notificações push aproveitando a PWA, importação de extratos bancários em CSV, scan QR Codes de faturas com criação de despesa/receita automática.

Nenhuma destas limitações é bloqueante. A aplicação está completa e em produção.